Soda.io Meets Spark

16 December 2025 15:31

Soda.io

→ A platform to run quality checks on data

Parts:

① SodaCL (Soda Checks Language)

→ Yaml that describes what "good data" means

```
yaml
checks for esg_ghg_estimates_20250101:
  - row_count > 0
  - missing_count (company_id) = 0
  - avg (ghg_estimate) between 0 and 50
  - duplicate_count Company_id, perid) = 0
```

② Warehouse / Data Source config

→ Tells soda how to reach your SQL engine.

For Spark:

```
yaml
data_source esg_spark:
    type: Spark
```

③ The Scan Engine (Scan object)

1 ↳ parses SodaCL
2 ↳ converts each check to SQL
3 ↳ Run SQL against the data source (eg: spark)
4 ↳ Collect results
5 ↳ Build json, logs, metrics, outcomes
6 ↳ Optionally return Fail/warn with diagnostics

```
python
scanner = Scan()
scanner.add_spark_session(spark_session, SODA_DATASOURCE)
scanner.add_sodacl_yaml_str (aggregated_yaml_str)
scanner.execute()
```

This is equivalent to

```
bash
soda scan -d spark -c warehouse.yml checks.yml
```

④ Sampler (optional)

handles:

- ✲ capturing failed rows
- ✲ showing sample values
- ✲ saving failed rows to cloud storage / somewhere else

Basically lets you inject custom python logic into Soda's execution.

But the python is not to define Soda rules; it is to generate soda rules, run soda, interpret Soda results, and attach Bloomberg specific logic.

✲ soda as a python library == "soda core"

⑤ Soda Cloud (optional)

SaaS dashboard

✲ Example in prod:

Soda reads the qc rules from a db, translates the rules into soda cl using python, then uses soda CL & the data from spark to run the soda CL on the spark data.

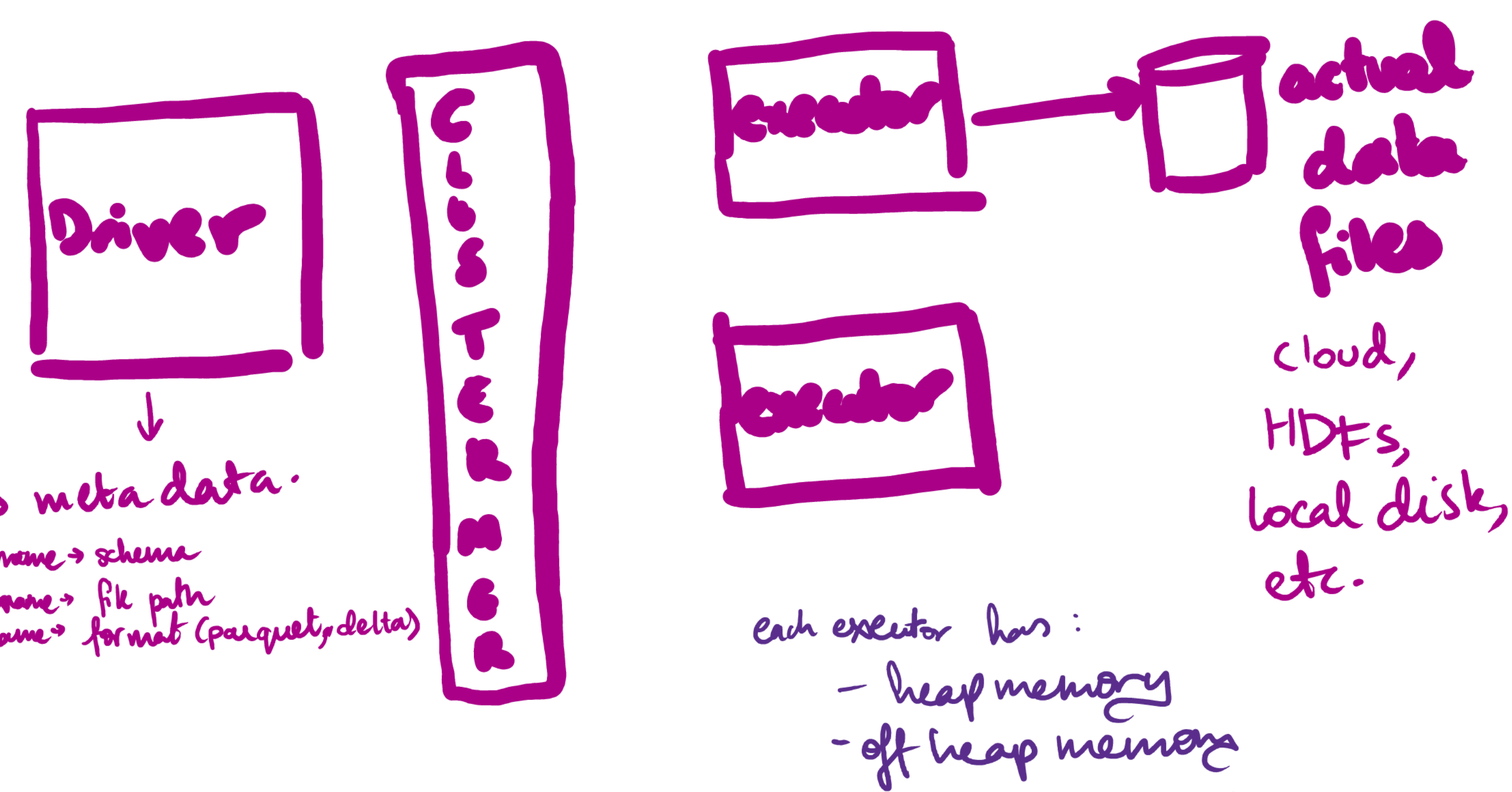
✓ helps with dynamic rules rather than static ones.

Spark Recap

→ An in memory processing engine.

→ ETL pipeline → Extract, Transform, Load

→ Not a Database, but a "data source" because from Soda's perspective, you can run SQL queries on the spark data.



↓ has meta data.

- tablename → schema
- tablename → file path
- tablename → format (parquet, delta)

each executor has:

- heap memory
- off heap memory
- cache / persist storage
- shuffle memory
- execution memory

✲ Tungsten → Spark's memory + execution engine

→ to avoid JVM object overhead.

→ off heap memory management

✲ when possible, Spark tries to use executors with cached data, but

- locality is best effort
- scheduler may choose other executors depending on availability.

Each executor's memory forms cluster memory, which is where data of temporary views are stored.

↓ These are created using 'createOrReplaceView'

They disappear when the spark job ends.

So, an underlying dataframe might be in executor memory or reconstructed from source files when queried.

Spark [illegible] Soda

Soda uses spark as a data store since it reads the views that ran as part of a spark job. E.g there was a spark job that ran that took a snapshot of some data. Soda will run quality checks on this data by using the data that is in spark.

why does soda go through spark & not use the database directly?

Spark job
↓
creates snapshot tables inside spark for the audit job.

They exist only inside spark for the audit job. they aren't in an external SQL db; just "spark views".

Could also be snapshots across many datastores.

✲ Soda can't read a pandas dataframe. It works with SQL datasources only.

✲ The Chain:

$$\text{Soda} \xrightarrow{\text{spark SQL}} \boxed{\text{spark}}$$

Spark → parses the SQL

- optimizes the SQL using the Catalyst Optimizer
- creates a distributed physical plan
- launches tasks on executors
- executors read files in parallel
- executors return data to driver
- Soda gets the result.